

Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Звіт

з лабораторної роботи № 5 з дисципліни
«Проектування алгоритмів»

„Проектування і аналіз алгоритмів для вирішення NP-складних задач ч.1”

Виконав(ла)

ІП-341 Черепов Олександр Павлович

(шифр, прізвище, ім'я, по батькові)

Перевірив

Головченко М.М.

(прізвище, ім'я, по батькові)

Київ 2025

ЗМІСТ

1	МЕТА ЛАБОРАТОРНОЇ РОБОТИ	3
2	ЗАВДАННЯ	4
3	ВИКОНАННЯ.....	11
3.1	ПРОГРАМНА РЕАЛІЗАЦІЯ АЛГОРИТМУ	11
3.1.1	<i>Вихідний код.....</i>	<i>11</i>
3.1.2	<i>Приклади роботи</i>	<i>14</i>
3.2	ТЕСТУВАННЯ АЛГОРИТМУ	15
3.2.1	<i>Значення цільової функції зі збільшенням кількості ітерацій .</i>	<i>15</i>
3.2.2	<i>Графіки залежності розв'язку від числа ітерацій</i>	<i>16</i>
	ВИСНОВОК	17
	КРИТЕРІЇ ОЦІНЮВАННЯ	17

1 МЕТА ЛАБОРАТОРНОЇ РОБОТИ

Мета роботи – вивчити основні підходи формалізації метаевристичних алгоритмів і вирішення типових задач з їхньою допомогою.

2 ЗАВДАННЯ

Згідно варіанту, розробити алгоритм вирішення задачі і виконати його програмну реалізацію на будь-якій мові програмування.

Задача, алгоритм і його параметри наведені в таблиці 2.1.

Зафіксувати якість отриманого розв'язку (значення цільової функції) після кожних 20 ітерацій до 1000 (допускається самостійний вибір кроку та верхньої границі ітерацій) і побудувати графік залежності якості розв'язку від числа ітерацій.

За допомогою чат-боту створити програму, яка точно розв'язує задачу з таблиці 2.1.

Зробити узагальнений висновок у якому порівняти дві версії програми.

Таблиця 2.1 – Варіанти алгоритмів

№	Задача і алгоритм
1	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 20 (випадкова), вага від 1 до 10 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування одноточковий по 50 генів, мутація з ймовірністю 5% змінюємо тільки 1 випадковий ген). Оператор локального покращення – пошук із сходженням до вершини (починає працювати з n -ї ітерації).
2	Задача комівояжера (100 вершин, відстань між вершинами випадкова від 5 до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 4$, $\rho = 0,4$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 30$, починають маршрут в різних випадкових вершинах).
3	Задача розфарбовування графу (200 вершин, степінь вершини не більше 20, але не менше 1), бджолиний алгоритм ABC (число бджіл 30 із них 2 розвідники).
4	Задача про рюкзак (місткість $P=200$, 100 предметів, цінність предметів від 2 до 20 (випадкова), вага від 1 до 10 (випадкова)),

	генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування двоточковий порівну генів, мутація з ймовірністю 10% змінюємо тільки 1 випадковий ген). Оператор локального покращення – пошук із емуляцією відпалу (починає працювати з n-ї ітерації).
5	Задача комівояжера (150 вершин, відстань між вершинами випадкова від 5 до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 3$, $\rho = 0,4$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 35$, починають маршрут в різних випадкових вершинах).
6	Задача розфарбовування графу (250 вершин, степінь вершини не більше 25, але не менше 2), бджолиний алгоритм ABC (число бджіл 35 із них 3 розвідники).
7	Задача про рюкзак (місткість $P=150$, 100 предметів, цінність предметів від 2 до 10 (випадкова), вага від 1 до 5 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування рівномірний, мутація з ймовірністю 5% два випадкові гени міняються місцями). Оператор локального покращення – променевий пошук (починає працювати з n-ї ітерації).
8	Задача комівояжера (200 вершин, відстань між вершинами випадкова від 0(перехід заборонено) до 50), мурашиний алгоритм ($\alpha = 3$, $\beta = 2$, $\rho = 0,3$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 45$, починають маршрут в різних випадкових вершинах).
9	Задача розфарбовування графу (150 вершин, степінь вершини не більше 30, але не менше 1), бджолиний алгоритм ABC (число бджіл 25 із них 3 розвідники).
10	Задача про рюкзак (місткість $P=150$, 100 предметів, цінність предметів від 2 до 10 (випадкова), вага від 1 до 5 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1

	різному предмету, оператор схрещування рівномірний, мутація з ймовірністю 10% два випадкові гени міняються місцями). Оператор локального покращення – пошук із сходженням до вершини (починає працювати з n-ї ітерації).
11	Задача комівояжера (250 вершин, відстань між вершинами випадкова від 0(перехід заборонено) до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 4$, $\rho = 0,6$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 45$, починають маршрут в різних випадкових вершинах).
12	Задача розфарбовування графу (300 вершин, степінь вершини не більше 30, але не менше 1), бджолиний алгоритм ABC (число бджіл 60 із них 5 розвідники).
13	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 30 (випадкова), вага від 1 до 25 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування одноточковий 30% і 70%, мутація з ймовірністю 5% два випадкові гени міняються місцями). Оператор локального покращення – пошук із емуляцією відпалу (починає працювати з n-ї ітерації).
14	Задача комівояжера (250 вершин, відстань між вершинами випадкова від 1 до 40), мурашиний алгоритм ($\alpha = 4$, $\beta = 2$, $\rho = 0,3$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 45$ (10 з них дикі, обирають випадкові напрямки), починають маршрут в різних випадкових вершинах).
15	Задача розфарбовування графу (100 вершин, степінь вершини не більше 20, але не менше 1), класичний бджолиний алгоритм (число бджіл 30 із них 3 розвідники).
16	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 30 (випадкова), вага від 1 до 25 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1

	різному предмету, оператор схрещування двоточковий 30%, 40% і 30%, мутація з ймовірністю 10% два випадкові гени міняються місцями). Оператор локального покращення – променевий пошук (починає працювати з n-ї ітерації).
17	Задача комівояжера (200 вершин, відстань між вершинами випадкова від 1 до 40), мурашиний алгоритм ($\alpha = 2$, $\beta = 4$, $\rho = 0,7$, $L_{\min}$ знайти жадібним алгоритмом, кількість мурах $M = 45$ (15 з них дикі, обирають випадкові напрямки), починають маршрут в різних випадкових вершинах).
18	Задача розфарбовування графу (300 вершин, степінь вершини не більше 50, але не менше 1), класичний бджолиний алгоритм (число бджіл 60 із них 5 розвідники).
19	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 30 (випадкова), вага від 1 до 25 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування триточковий 25%, мутація з ймовірністю 5% два випадкові гени міняються місцями). Оператор локального покращення – пошук із сходженням до вершини (починає працювати з n-ї ітерації).
20	Задача комівояжера (200 вершин, відстань між вершинами випадкова від 1 до 40), мурашиний алгоритм ($\alpha = 3$, $\beta = 2$, $\rho = 0,7$, $L_{\min}$ знайти жадібним алгоритмом, кількість мурах $M = 45$ (10 з них елітні, подвійний феромон), починають маршрут в різних випадкових вершинах).
21	Задача розфарбовування графу (200 вершин, степінь вершини не більше 30, але не менше 1), класичний бджолиний алгоритм (число бджіл 40 із них 2 розвідники).
22	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 30 (випадкова), вага від 1 до 25 (випадкова)),

	генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування триточковий 25%, мутація з ймовірністю 5% змінюємо тільки 1 випадковий ген). Оператор локального покращення – пошук із емуляцією відпалу (починає працювати з n-ї ітерації).
23	Задача комівояжера (300 вершин, відстань між вершинами випадкова від 1 до 60), мурашиний алгоритм ($\alpha = 3$, $\beta = 2$, $\rho = 0,6$, Lmin знайти жадібним алгоритмом, кількість мурах $M = 45$ (15 з них елітні, подвійний феромон), починають маршрут в різних випадкових вершинах).
24	Задача розфарбовування графу (400 вершин, степінь вершини не більше 50, але не менше 1), класичний бджолиний алгоритм (число бджіл 70 із них 10 розвідники).
25	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 20 (випадкова), вага від 1 до 10 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування одноточковий по 50 генів, мутація з ймовірністю 5% змінюємо тільки 1 випадковий ген). Оператор локального покращення – променевий пошук (починає працювати з n-ї ітерації).
26	Задача комівояжера (100 вершин, відстань між вершинами випадкова від 5 до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 4$, $\rho = 0,4$, Lmin знайти жадібним алгоритмом, кількість мурах $M = 30$, починають маршрут в різних випадкових вершинах).
27	Задача розфарбовування графу (200 вершин, степінь вершини не більше 20, але не менше 1), бджолиний алгоритм ABC (число бджіл 30 із них 2 розвідники).
28	Задача про рюкзак (місткість $P=200$, 100 предметів, цінність предметів від 2 до 20 (випадкова), вага від 1 до 10 (випадкова)),

	генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування двоточковий порівну генів, мутація з ймовірністю 10% змінюємо тільки 1 випадковий ген). Оператор локального покращення – пошук із сходженням до вершини (починає працювати з n-ї ітерації).
29	Задача комівояжера (150 вершин, відстань між вершинами випадкова від 5 до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 3$, $\rho = 0,4$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 35$, починають маршрут в різних випадкових вершинах).
30	Задача розфарбовування графу (250 вершин, степінь вершини не більше 25, але не менше 2), бджолиний алгоритм ABC (число бджіл 35 із них 3 розвідники).
31	Задача про рюкзак (місткість $P=250$, 100 предметів, цінність предметів від 2 до 20 (випадкова), вага від 1 до 10 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1 різному предмету, оператор схрещування одноточковий по 50 генів, мутація з ймовірністю 5% змінюємо тільки 1 випадковий ген). Оператор локального покращення – пошук із емуляцією відпалу (починає працювати з n-ї ітерації).
32	Задача комівояжера (100 вершин, відстань між вершинами випадкова від 5 до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 4$, $\rho = 0,4$, L_{min} знайти жадібним алгоритмом, кількість мурах $M = 30$, починають маршрут в різних випадкових вершинах).
33	Задача розфарбовування графу (200 вершин, степінь вершини не більше 20, але не менше 1), бджолиний алгоритм ABC (число бджіл 30 із них 2 розвідники).
34	Задача про рюкзак (місткість $P=200$, 100 предметів, цінність предметів від 2 до 20 (випадкова), вага від 1 до 10 (випадкова)), генетичний алгоритм (початкова популяція 100 осіб кожна по 1

	різному предмету, оператор схрещування двоточковий порівню генів, мутація з ймовірністю 10% змінюємо тільки 1 випадковий ген). Оператор локального покращення – променевий пошук (починає працювати з n-ї ітерації).
35	Задача комівояжера (150 вершин, відстань між вершинами випадкова від 5 до 50), мурашиний алгоритм ($\alpha = 2$, $\beta = 3$, $\rho = 0,4$, Lmin знайти жадібним алгоритмом, кількість мурах $M = 35$, починають маршрут в різних випадкових вершинах).

3.1 Програмна реалізація алгоритму

3.1.1 Вихідний код

main.py:

```

from generator import generate_items, generate_population
from interfaces import Individual
import math
import random
import copy

CAPACITY = 250
NUM_ITEMS = 100
POPULATION_SIZE = 100
CROSSOVER_RATE = 0.25
MUTATION_RATE = 0.05
MAX_ITERATIONS = 1000
SA_START_ITERATION = 200

def calculate_individual_value(individual, items):
    total_v = 0
    total_w = 0

    for i in range(NUM_ITEMS):
        if individual.genes[i] == 1:
            total_v += items[i]["v"]
            total_w += items[i]["w"]

    individual.weight = total_w
    if total_w > CAPACITY:
        individual.value = 0
    else:
        individual.value = total_v

def crossover(parent1, parent2):
    if random.random() > CROSSOVER_RATE:
        return copy.deepcopy(parent1)

    points = sorted(random.sample(range(1, NUM_ITEMS), 3))
    k1, k2, k3 = points

    child_genes = parent1.genes[:k1] + parent2.genes[k1:k2] +
parent1.genes[k2:k3] + parent2.genes[k3:]

    return Individual(child_genes)

```

```

def mutate(individual, items):
    if random.random() < MUTATION_RATE:
        index = random.randint(0, NUM_ITEMS - 1)
        individual.genes[index] = 1 - individual.genes[index]
        calculate_individual_value(individual, items)

def simulated_annealing(individual, current_iter, items):
    T = 50.0 / (1 + 0.05 * (current_iter - SA_START_ITERATION))

    neighbor_genes = individual.genes[:]
    index = random.randint(0, NUM_ITEMS - 1)
    neighbor_genes[index] = 1 - neighbor_genes[index]
    neighbor = Individual(neighbor_genes)

    calculate_individual_value(neighbor, items)
    delta = neighbor.value - individual.value

    if delta > 0:
        individual.genes = neighbor_genes
        individual.value = neighbor.value
        individual.weight = neighbor.weight
    else:
        if T > 0.001:
            prob = math.exp(delta / T)
            if random.random() < prob:
                individual.genes = neighbor_genes
                individual.value = neighbor.value
                individual.weight = neighbor.weight

def main():
    items = generate_items(NUM_ITEMS)
    population = generate_population(POPULATION_SIZE, NUM_ITEMS)

    best_global_value = 0

    print(f"{'Iteration':<15} | {'Best value':<15} | {'Current weight':<15}")

    for iteration in range(MAX_ITERATIONS + 1):
        population.sort(key=lambda x: x.value, reverse=True)

        current_best = population[0]
        if current_best.value > best_global_value:
            best_global_value = current_best.value

        if iteration % 20 == 0:
            print(f"{iteration:<15} | {current_best.value:<15} | {current_best.weight}/{CAPACITY:<15}")

        new_population = copy.deepcopy(population[:10])

```

```

        while len(new_population) < POPULATION_SIZE:
            parent1 = max(random.sample(population, 5), key=lambda x:
x.value)
            parent2 = max(random.sample(population, 5), key=lambda x:
x.value)

            child = crossover(parent1, parent2)
            calculate_individual_value(child, items)
            mutate(child, items)

            if iteration >= SA_START_ITERATION:
                simulated_annealing(child, iteration, items)

            new_population.append(child)

        population = new_population

    print("-" * 50)
    print(f"Result: {best_global_value}\n")

main()

```

generate.py:

```

import random
from interfaces import Individual

def generate_items(num_items, seed=-1):
    if seed != -1:
        random.seed(seed)

    items = []
    for _ in range(num_items):
        items.append({
            "v": random.randint(2, 30),
            "w": random.randint(1, 25)
        })

    return items

def generate_population(population_size, gen_size):
    population = [Individual() for _ in range(population_size)]

    for index, individual in enumerate(population):
        genes = [0] * gen_size

```

```

    if index < gen_size:
        genes[index] = 1
    individual.genes = genes

return population

```

interfaces.py:

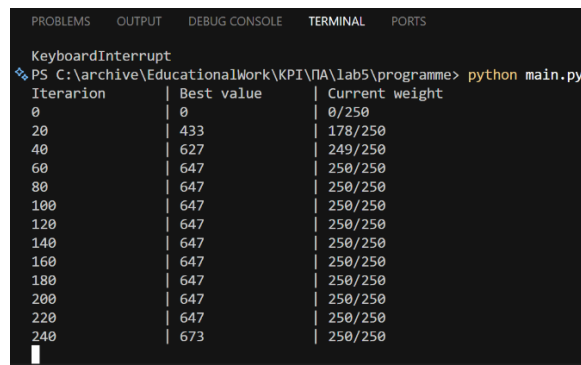
```

class Individual:
    def __init__(self, genes=None):
        self.value = 0
        self.weight = 0
        self.genes = genes

```

3.1.2 Приклади роботи

На рисунках 3.1 і 3.2 показані приклади роботи програми.

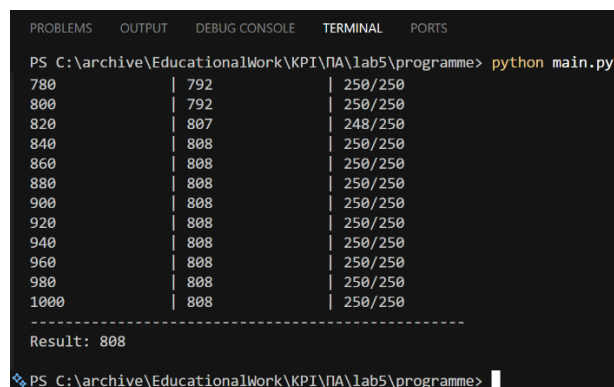


```

KeyboardInterrupt
PS C:\archive\EducationalWork\KPI\ПА\lab5\programme> python main.py
Iteration | Best value | Current weight
0         | 0         | 0/250
20        | 433       | 178/250
40        | 627       | 249/250
60        | 647       | 250/250
80        | 647       | 250/250
100       | 647       | 250/250
120       | 647       | 250/250
140       | 647       | 250/250
160       | 647       | 250/250
180       | 647       | 250/250
200       | 647       | 250/250
220       | 647       | 250/250
240       | 673       | 250/250

```

Рисунок 3.1 – Процес виконання програми



```

PS C:\archive\EducationalWork\KPI\ПА\lab5\programme> python main.py
780       | 792       | 250/250
800       | 792       | 250/250
820       | 807       | 248/250
840       | 808       | 250/250
860       | 808       | 250/250
880       | 808       | 250/250
900       | 808       | 250/250
920       | 808       | 250/250
940       | 808       | 250/250
960       | 808       | 250/250
980       | 808       | 250/250
1000      | 808       | 250/250
-----
Result: 808
PS C:\archive\EducationalWork\KPI\ПА\lab5\programme>

```

Рисунок 3.2 – Результат виконання програми

3.2 Тестування алгоритму

Значення цільової функції зі збільшенням кількості ітерацій

У таблиці 3.1 наведено значення цільової функції зі збільшенням кількості ітерацій.

Таблиця 3.1

Ітерація	Значення
0	0
20	433
40	627
60	647
80	647
100	647
120	647
140	647
160	647
180	647
200	647
220	647
240	673
260	673
280	701
300	701
320	733
340	782
360	782
380	782
400	791
420	791
440	791
460	791
480	791
500	791
520	791
540	791
560	791
580	791
600	792

Продовження Таблиці 3.1

620	792
640	792
660	792
680	792
700	792
720	792
740	792
760	792
780	792
800	792
820	807
840	808
860	808
880	808
900	808
920	808
940	808
960	808
980	808
1000	808

3.2.1 Графіки залежності розв'язку від числа ітерацій

На рисунку 3.3 наведений графік, який показує якість отриманого розв'язку.



Рисунок 3.3 – Графіки залежності розв'язку від числа ітерацій

ВИСНОВОК

В рамках даної лабораторної роботи було розроблено та досліджено гібридний генетичний алгоритм для розв'язання задачі про рюкзак. В ході тестування встановлено пряму залежність якості розв'язку від тривалості роботи алгоритму: максимізація цільової функції досягається при збільшенні кількості ітерацій, що підтверджується графіком збіжності.

Також експериментально виявлено, що зменшення порогу для активації імітації відпалу позитивно впливає на результат, дозволяючи алгоритму раніше уникати локальних екстремумів та знаходити кращі комбінації предметів.

Порівняння з точним методом, запропонованим чат-ботом на основі моделі Claude Sonnet 4.5 (Free plan), який показав результат 869, виявило, що розроблений метаевристичний алгоритм знайшов рішення 808. Це значення є достатньо близьким до ідеального, що підтверджує ефективність генетичних підходів для вирішення складних задач.

Критерії оцінювання

Максимальний бал за виконання лабораторної роботи дорівнює – 5.

Критерії оцінювання:

- проектування алгоритму – 1;
- програмна реалізація алгоритму – 1;
- захист – 1;
- дослідження алгоритму – 1;
- звіт – 1.